INFORMATIONSBLATT

IB03 · Cronjobs – Zeitgesteuerte Automatisierung

LF10b · LS03 – Administrationsaufgaben mit Bash automatisieren · IB03 – Cronjobs: Zeitgesteuerte Automatisierung | IT-3B · Schuljahr 2025/2026

1 · Was ist Cron?

📖 **Cron-Daemon**
Cron ist ein Systemdienst (Daemon) unter Linux, der Befehle oder Skripte zu festgelegten Zeitpunkten automatisch ausführt. Er läuft im Hintergrund und liest regelmäßig die **Crontab-Dateien**, um zu prüfen, ob eine Aufgabe ausgeführt werden muss.

Der Name stammt von „Chronos" (griechisch: Zeit). Der Daemon heißt `cron`, der Befehl zum Verwalten der Zeitplan-Dateien heißt `crontab`.

Cron ist auf Ubuntu standardmäßig aktiv. Sie können den Status prüfen:

Cron-Status prüfen

```
systemctl status cron
```

2 · Die Cron-Syntax

2.1 | Aufbau eines Crontab-Eintrags

Jeder Eintrag in einer Crontab-Datei besteht aus **sechs Teilen**: fünf Zeitfeldern und dem auszuführenden Befehl.

```
┌──────────── Minute        (0–59)
│  ┌──────────── Stunde        (0–23)
│  │  ┌──────────── Tag des Monats (1–31)
│  │  │  ┌──────────── Monat          (1–12 oder Jan–Dec)
│  │  │  │  ┌──────────── Wochentag       (0–7, wobei 0 und 7 = Sonntag)
│  │  │  │  │
*  *  *  *  *   /absoluter/pfad/zum/skript.sh
```

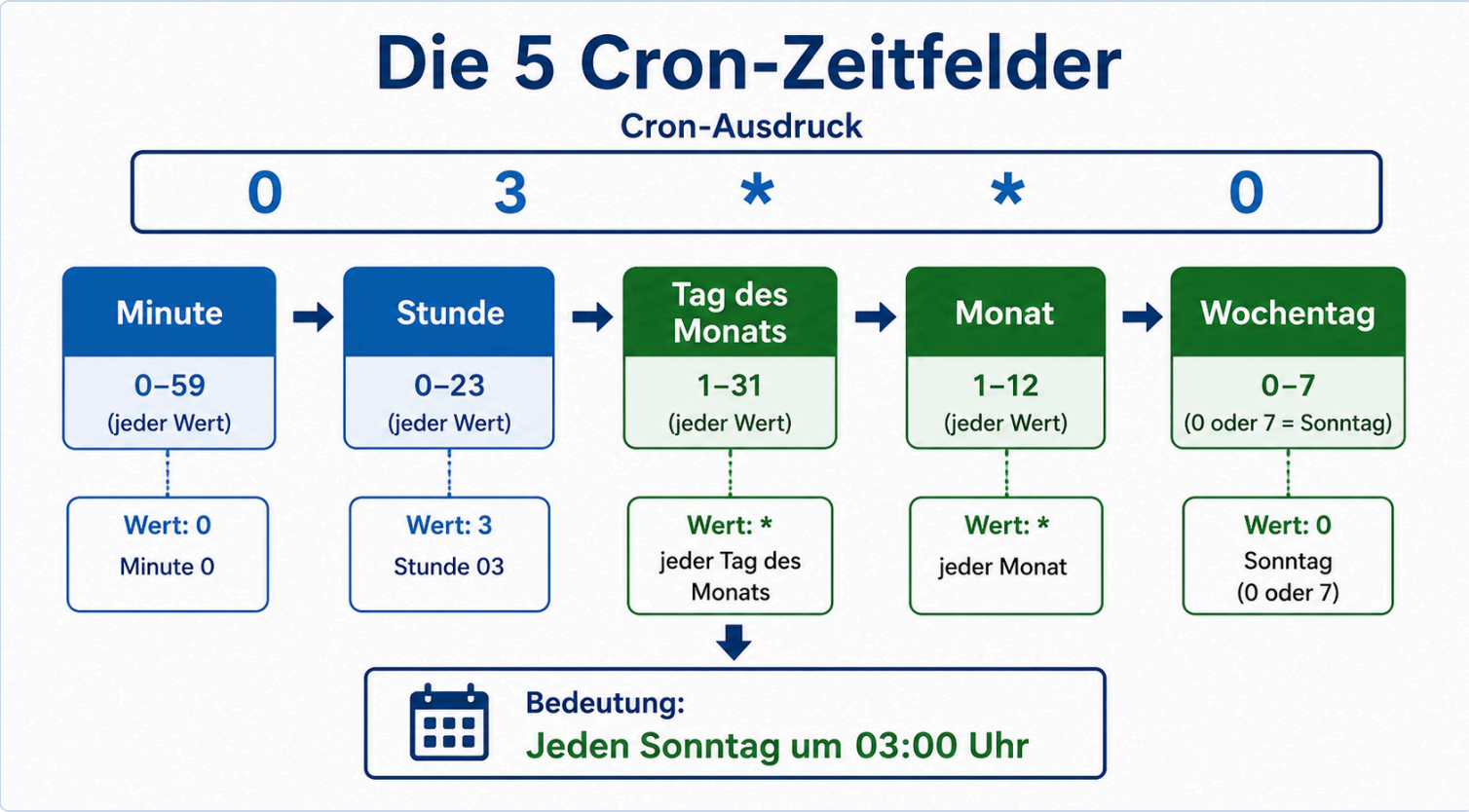


Abbildung 1: Cron-Zeitfelder – Aufbau und Wertebereich am Beispiel `0 3 * * 0` (jeden Sonntag um 03:00 Uhr)

2.2 | Sonderzeichen in Zeitfeldern

Zeichen	Bedeutung	Beispiel	Erklärung
`*`	Jeden Wert	`* * * * *`	Jede Minute
`5`	Exakter Wert	`0 5 * * *`	Täglich um 05:00 Uhr
`1-5`	Bereich	`0 8 * * 1-5`	Mo–Fr um 08:00 Uhr
`*/5`	Schrittweite	`*/5 * * * *`	Alle 5 Minuten
`1,15`	Liste	`0 2 1,15 * *`	Am 1. und 15. jeden Monats um 02:00

2.3 | Praxisbeispiele – Cron-Ausdrücke lesen

Cron-Ausdruck	Bedeutung
`0 3 * * *`	Täglich um 03:00 Uhr
`0 4 * * 0`	Jeden Sonntag um 04:00 Uhr
`*/5 * * * *`	Alle 5 Minuten (rund um die Uhr)
`30 6 * * 1-5`	Montag bis Freitag um 06:30 Uhr
`0 0 1 * *`	Am ersten Tag jedes Monats um Mitternacht
`0 */6 * * *`	Alle 6 Stunden (0, 6, 12, 18 Uhr)
`15 2 * * 6`	Jeden Samstag um 02:15 Uhr

✅ **Tipp: Cron-Ausdrücke online testen**
Unter **crontab.guru** können Sie Cron-Ausdrücke in verständlicher Sprache anzeigen lassen und neue Ausdrücke interaktiv entwickeln. Nutzen Sie das Tool zur Überprüfung Ihrer Einträge – vor allem bei komplexen Zeitplänen.

✅ **Zwischenübung – Cron-Ausdrücke selbst bilden**
Formulieren Sie für die folgenden Anforderungen den passenden Cron-Ausdruck. Schreiben Sie jeweils das vollständige Zeitfeld (alle 5 Felder):

Anforderung	Ihr Cron-Ausdruck (5 Felder)
Jeden Werktag (Mo–Fr) um 07:30 Uhr	
Jeden ersten des Monats um Mitternacht (00:00)	
Alle 10 Minuten, rund um die Uhr	
Jeden Samstag und Sonntag um 08:00 Uhr	

Tipp: Nutzen Sie *crontab.guru* zur Selbstkontrolle.

3 · Die Crontab bearbeiten

3.1 | Crontab-Befehle

Befehl	Aktion
`crontab -e`	Crontab des aktuellen Benutzers im Editor öffnen
`sudo crontab -e`	Crontab des Root-Benutzers bearbeiten
`crontab -l`	Aktuelle Crontab anzeigen (ohne Bearbeiten)
`sudo crontab -l`	Root-Crontab anzeigen
`crontab -r`	Crontab des aktuellen Benutzers löschen (Vorsicht!)

⚠️ **crontab -r ohne Bestätigung**
Der Befehl `crontab -r` löscht die gesamte Crontab *sofort und ohne Rückfrage*. Verwechslungen mit `crontab -e` können passieren. Erstellen Sie vorher immer ein Backup: `crontab -l > ~/crontab_backup.txt`

3.2 | Editor beim ersten Aufruf wählen

Beim ersten Aufruf von `crontab -e` auf einem neuen System werden Sie nach dem bevorzugten Editor gefragt. Wählen Sie **nano** (Option 1) – er ist für Einsteiger am einfachsten zu bedienen.

Editor-Auswahl beim ersten Aufruf

```
no crontab for root - using an empty one
Select an editor.  To change later, run 'select-editor'.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.tiny
  3. /usr/bin/vim

Choose 1-3 [1]: 1    ← Eingabe: 1 → Enter
```

4 · Vollständige Crontab-Einträge erstellen

4.1 | Aufbau eines korrekten Eintrags

Ein vollständiger Crontab-Eintrag besteht aus dem Cron-Ausdruck, dem auszuführenden Befehl und der Ausgabeumleitung in eine Logdatei.

Vollständiger Crontab-Eintrag mit Logging

```
# Erklärung:  Minute  Stunde  Tag  Monat  Wochentag  Befehl
# Backup täglich um 03:00 Uhr
0 3 * * * /opt/scripts/backup.sh >> /var/log/netconsult/cron.log 2>&1

# Log-Cleanup jeden Sonntag um 04:00 Uhr
0 4 * * 0 /opt/scripts/log_cleanup.sh >> /var/log/netconsult/cron.log 2>&1

# Apache-Monitoring alle 5 Minuten
*/5 * * * * /opt/scripts/monitor_apache.sh >> /var/log/netconsult/cron.log 2>&1
```

📖 **Ausgabeumleitung im Crontab-Eintrag**
Cronjobs haben standardmäßig keine Verbindung zu einem Terminal. Ohne Umleitung wird die Ausgabe als E-Mail an den Systembenutzer gesendet – was in der Praxis meist unerwünscht ist.

- `>> /var/log/datei.log` – normale Ausgabe anhängen
- `2>&1` – Fehlerausgabe (stderr) ebenfalls in dieselbe Datei
- `> /dev/null 2>&1` – alle Ausgaben vollständig unterdrücken

Für Produktionsumgebungen empfiehlt sich immer die Umleitung in eine Logdatei.

4.2 | Absolute Pfade verwenden

Cron läuft in einer minimalen Umgebung: Das Arbeitsverzeichnis ist das Home-Verzeichnis und `$PATH` enthält nur `/usr/bin:/bin` – nicht die gewohnten Pfade eines Benutzers (z. B. `/usr/sbin`). Geben Sie deshalb in Crontab-Einträgen und in Ihren Skripten **absolute Pfade** an – dann funktioniert der Job unabhängig von der Umgebung.

Unsicher in Cron	Problem	Richtig (absoluter Pfad)
`backup.sh`	Skript wird nicht gefunden (relativer Aufruf)	`/opt/scripts/backup.sh`
`apache2ctl configtest`	liegt in `/usr/sbin` – nicht im Cron-`$PATH` → „command not found"	`/usr/sbin/apache2ctl configtest`
`tar -czf archiv.tar.gz daten/`	relativer Datenpfad bezieht sich auf das Home-Verzeichnis	`tar -czf /backup/web/archiv.tar.gz /var/www/html/netconsult`

✅ Tipp: Pfade von Befehlen herausfinden

```
which tar          # Ausgabe: /usr/bin/tar
which systemctl    # Ausgabe: /usr/bin/systemctl
which apache2ctl   # Ausgabe: /usr/sbin/apache2ctl
```

Alternativ: Setzen Sie am Anfang der Crontab die PATH-Variable:

```
PATH=/usr/bin:/bin:/usr/sbin:/sbin
*/5 * * * * /opt/scripts/monitor_apache.sh
```

5 | Cronjob manuell testen

Bevor Sie auf den nächsten automatischen Ausführungszeitpunkt warten, testen Sie Ihre Skripte immer **manuell** aus der Shell:

```
Skripte vor dem Einrichten des Cronjobs testen

# Skript direkt ausführen und Ausgabe prüfen
sudo /opt/scripts/backup.sh

# Exit-Code prüfen
echo "Exit-Code: $?"

# Logdatei sofort kontrollieren
tail -20 /var/log/netconsult/backup.log
```

✅ Empfohlene Testreihenfolge

1. Skript manuell als Root ausführen (`sudo /opt/scripts/skript.sh`)
2. Logdatei prüfen (`cat` oder `tail`)
3. Exit-Code kontrollieren (`echo $?` → muss `0` sein)
4. Crontab-Eintrag anlegen
5. Nach dem nächsten Ausführungszeitpunkt im Syslog prüfen (z. B. `grep CRON /var/log/syslog`)
